Frontiers of Intelligence

Filippo Lentoni

May 24, 2026

Contents

About the Author

Filippo is a Senior Applied Scientist at Amazon, specialized in agentic planning and large-scale, compliance-critical machine learning systems. He is a visiting lecturer at Columbia Engineering, where he teaches theoretical introduction to AI Agents, and has collaborated with the Technical University of Munich as Science Lead on post-training research since 2023. Filippo holds a Master's degree in Mathematical Engineering and Statistical Learning from Politecnico di Milano. His work sits at the intersection of statistical learning theory, agentic AI, and the design of large-scale AI systems.

The idea for this book emerged from a conversation with Sherwin Wu, OpenAI's Head of Engineering, at UC Berkeley in 2025. Sherwin argued that despite the investment behind Operator, Deep Research, the Responses API, o3, and Codex, industry adoption of agentic AI remained limited. The core insight was that the primary blockers were not only intelligence-related, but deployment-related: models were becoming increasingly capable, yet the engineering infrastructure, evaluation discipline, and documentation required to deploy agents reliably were still immature. This book is an attempt to address that gap.

Chapter 1

The Anatomy of an Agent

This book begins with a simple question: what is an AI agent? The anatomy of an agentic AI system comprises four components: the intelligence layer, the action layer, the memory and knowledge layer, and the security layer. These layers are conceptually distinct but continuously interdependent in production: the model reasons over context assembled by the memory layer, selects actions exposed by the action layer, and operates within boundaries enforced by the security layer. Evaluation and observability sit across all four, measuring whether the full system behaves correctly at each step and over time.

1.1 The Four Layers of an Agent

The first layer is the *intelligence layer*. In this book, the intelligence layer encompasses two tightly coupled components: the foundation model, which provides the core reasoning and generation capability, and the cognitive architecture, which governs how the model pursues a goal across multiple steps, deciding when to act, when to retrieve, when to seek clarification, and when to stop. The subsequent chapter explains how these models work, beginning with the original Transformer architecture and progressing toward the design choices that characterise modern frontier models.

The second layer is the *action layer*. This layer enables the agent to produce effects beyond generating text. It encompasses function calling, tools, MCP servers and agent-as-tool patterns. Some tools are deterministic, such as a calculator, while others are stochastic, such as a sub-agent that performs research, writes code, or produces a judgment. Together they define the set of effects the agent can produce in the world beyond generating text.

The third layer is the *memory and knowledge layer*. This layer determines what information is available to the model at the moment of decision. It includes short-term conversation state, long-term user memory, retrieval-augmented generation, knowledge bases and reusable skills. In this book, skills are treated as a form of procedural knowledge: versioned, retrievable packages of instructions or patterns that improve the reliability with which the agent performs recurring tasks.

The fourth layer is the *security layer*. An agent with access to tools, persistent memory, and the ability to take actions requires security as a foundational architectural concern. The security layer includes identity, authentication, authorisation, permission boundaries, guardrails, human approval mechanisms, audit logging, sandboxing, and protection against prompt injection and data exfiltration. The scope and rigour of this layer should scale with the autonomy and consequence of the agent's permitted actions.

1.2 From Anatomy to Implementation

The architecture described above provides the conceptual foundation for the rest of the book. The chapters that follow develop each layer in depth: how foundation models are trained and how they reason, how the action layer is designed and governed, how the memory and knowledge layer is constructed and evaluated, and how the security layer is enforced in production. The application section then demonstrates how these layers compose into an end-to-end system: an agent built with LangGraph, connected to tools and memory, evaluated with Langfuse, and iterated through a pipeline that closes the loop between offline evaluation, production traces, human feedback, and future evaluation datasets. The goal is to give the reader the conceptual and practical tools to build an agent that can be tested, monitored, improved, and deployed through a governed production pipeline in their own domain.

Part I

The Intelligence Layer

Chapter 2

Foundation Models

The Transformer architecture is the substrate on which modern language models and therefore agentic AI systems are built. This chapter explains the modelling foundations needed for the rest of the book: how text becomes tokens, how tokens become vectors, and why decoder-only Transformers became the default architecture for general-purpose assistants.

2.1 Tokenisation: the First Compression Step

Before a model can process text, that text must be converted into tokens: discrete units such as words, subwords, bytes, or characters. Tokenisation shapes cost, context length, and multilingual behaviour.

Subword tokenisation, implemented through algorithms such as Byte-Pair Encoding, is the approach used by most modern language models. Common words are represented as a single token; rare words are decomposed into reusable subword pieces; and out-of-vocabulary terms are handled gracefully by decomposing them into familiar subword units, balancing vocabulary size against sequence length more effectively than word-level or character-level alternatives.

For agent builders, tokenisation has direct engineering consequences: the cost and context capacity of any operation scale with token count.

2.2 Embeddings and Representation Geometry

After tokenisation, each token is mapped to a dense vector called an embedding. Dense embeddings place tokens in a learned vector space where similar meanings tend to occupy related regions, expressing semantic relatedness as geometric proximity.

The same principle underlies retrieval. A document chunk and a query can both be embedded into the same vector space, and similarity in that space selects relevant context. Both language models and retrieval-augmented generation systems rest on the assumption that meaning can be approximated by geometry in a learned representation space.

2.3 The Limitations of Sequential Models

Before Transformers, recurrent models such as RNNs and LSTMs processed text sequentially, updating a hidden state token by token. This introduced order sensitivity but imposed two significant constraints: long-range dependencies were difficult to maintain as information from early tokens

could fade across the sequence, and training was difficult to parallelise because later hidden states depended on earlier ones.

2.4 Self-Attention

Self-attention allows each token to directly compare itself to every other token in the sequence. For each position, the model constructs three learned projections: a query, a key, and a value, where the query represents what the token is looking for, the key represents what another token offers for matching, and the value represents the content to be aggregated.

Scaled dot-product attention is defined as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

The matrix $QK^\top$ contains pairwise relevance scores between query and key positions; the softmax converts those scores into a probability distribution; multiplying by V produces a weighted combination of value vectors; and the division by $\sqrt{d_k}$ stabilises the scale of dot products as dimensionality grows.

Multi-head attention repeats this operation with multiple learned projections in parallel, allowing the model to attend simultaneously to local syntactic structure, entity references, and long-range answer spans, producing representations that are context-aware across multiple dimensions at once.

2.5 The Original Transformer

The original Transformer, introduced in *Attention Is All You Need*, used an encoder-decoder architecture. The encoder processed the source sequence with full bidirectional self-attention; the decoder generated the target sequence autoregressively through masked self-attention over prior outputs and cross-attention over encoder representations.

The architecture established the design patterns that remain central to modern models: attention sublayers for token-to-token interaction, position-wise feedforward sublayers for nonlinear transformation, residual connections for gradient flow, normalisation for training stability, positional encodings to supply order information that permutation-invariant attention cannot recover, and causal masking to enforce left-to-right generation. Together these choices gave every token a direct path to every other token and expressed the computation as matrix operations that GPUs can parallelise efficiently.

2.6 From Encoder-Decoder to Decoder-Only Models

Three Transformer families emerged from the original design. Encoder-only models such as BERT apply bidirectional self-attention to produce strong contextual representations for classification, extraction, and ranking. Encoder-decoder models such as T5 retain the full sequence-to-sequence structure for tasks requiring one sequence to be transformed into another. Decoder-only models became dominant for large generative models because next-token prediction is simple, scalable, and supports open-ended generation.

Decoder-only models are the architecture of choice for agent systems because a conversation, a plan, a tool call, a tool result, and a final response can all be serialised into a single sequence that the model learns to continue in the desired format.

2.7 Positional Information

Self-attention is permutation-invariant: the same scores result regardless of token order unless positional information is explicitly introduced. Early Transformers added absolute positional encodings to token embeddings; modern frontier models use Rotary Positional Embeddings (RoPE), which encode position through the attention computation itself and generalise more effectively to long sequences.

2.8 Inference: Logits, Decoding, and Structured Output

At inference time, a decoder-only model produces a distribution over the vocabulary at each step as a vector of logits. Greedy decoding selects the highest-probability token; sampling introduces stochasticity controlled by temperature, top-k, and top-p parameters.

2.9 Why This Matters for Agents

Every agentic capability is built on these modelling primitives: tool calls are generated token sequences, retrieved documents and memory summaries are context tokens, plans are generated text, and cost and latency scale with total token count. Understanding Transformers at this level supports the engineering decisions that follow in every subsequent chapter: how to manage context, when to constrain generation, why retrieval precision matters, and how model capability translates into agent reliability.

2.10 Summary

Transformers replaced sequential recurrence with parallel self-attention, giving every token a direct path to every other token in a form that modern hardware can execute efficiently. Decoder-only Transformers became the default for large generative models because next-token prediction scales cleanly and supports open-ended generation. Agents are built by placing this generative engine inside a stateful loop with tools, memory, context engineering, and security boundaries, the layers this book addresses in the chapters that follow.

Chapter 3

Frontier Model Capabilities

A frontier model is a model near the leading edge of capability at a given moment. The term refers to proximity to the current frontier across one or more dimensions: reasoning, coding, tool use, multimodality, long-context understanding, instruction following, safety behaviour, latency.

For agent builders, frontier models matter because they determine which workflows can be automated with acceptable reliability.

For example, for agentic coding, SWE-bench evaluates whether a frontier model can resolve real GitHub issues by modifying code and passing the repository's own test suite.

3.1 Open-Weight and Closed Models

The model ecosystem is often described as a competition between open and closed models, but the terminology requires precision. Closed or proprietary models are those whose weights are withheld from public release. Users access them through an API or hosted product, and the provider controls serving, safety behaviour, updates, and pricing. Open-weight models are those whose weights are released, typically under a licence. Users can inspect, host, quantise, and in many cases fine-tune them.

Closed models are often the most direct path to frontier capability. Open-weight models offer control, data privacy, cost optimisation, and customisation. In practice, many deployments use both: a closed model for reasoning-intensive paths and an open-weight model for high-volume internal workflows where cost and latency are primary constraints.

3.2 The Original Transformer

The architectural foundation worth understanding is the original Transformer, introduced by Vaswani et al. in 2017. It comprised an encoder and a decoder, both built from multi-head self-attention, position-wise feedforward networks, residual connections, layer normalisation, and positional encodings. The encoder processed the full input sequence with bidirectional self-attention. The decoder generated output autoregressively, using masked self-attention over its own prior outputs and cross-attention over the encoder's representations.

The original design targeted sequence-to-sequence tasks such as machine translation rather than general-purpose language modelling. Its lasting contribution was the attention mechanism as a content-dependent communication primitive: queries and keys determine where the model attends; values carry the content aggregated at each position. This separation of relevance from

content is what allows a later token to use an earlier definition, a function call to attend to its declaration site, or a question to locate its answer span across a long document.

3.3 The Modern Frontier Decoder

Contemporary frontier models are almost universally decoder-only, but the internal block has been substantially refined from the original design. The field has converged on a representative set of architectural components, reflected across both closed models such as GPT, Claude, and Gemini, and open-weight families such as Llama, Qwen, Mistral, and DeepSeek.

Causal self-attention replaces the encoder-decoder cross-attention structure, with the model attending only to prior positions during generation. Rotary Positional Embeddings (RoPE) encode position through rotation applied to query and key vectors rather than through additive absolute embeddings, generalising more effectively to sequence lengths beyond those seen during training. RMSNorm, applied before each sub-layer rather than after, provides training stability with lower computational overhead than the original LayerNorm. Grouped-Query Attention, which has become the default for most large models, shares key and value projections across groups of query heads, substantially reducing the memory footprint of the KV cache during inference without significant loss in model quality. SwiGLU, combining a Swish activation with a gated linear unit, has replaced earlier activation functions in the feedforward sub-layer and is now the de facto standard. FlashAttention-style kernels reorganise the attention computation to minimise memory movement between GPU SRAM and HBM, reducing both memory usage and wall-clock time. Sparse Mixture-of-Experts feedforward layers activate only a subset of specialised sub-networks for each token, allowing a large total parameter count with a much smaller active parameter count per forward pass. Multimodal encoders or adapters extend the decoder to accept image, audio, or video inputs alongside text.

3.4 How the Field Evolved

The trajectory from the original Transformer to current agentic deployments can be read as a sequence of distinct eras, each expanding the scope of what language models could do.

The representation era was defined by encoder-only models such as BERT, which produced strong contextual embeddings through bidirectional attention and achieved state-of-the-art results on classification, extraction, and ranking tasks. These models established the value of deep contextual representations but were designed for understanding rather than generation.

The text-to-text era unified diverse tasks under a single sequence-to-sequence interface. Models such as T5 reframed translation, summarisation, question answering, and classification as instances of the same text-in, text-out paradigm, demonstrating that a single model architecture and training objective could cover a broad range of downstream applications.

The autoregressive scaling era established that next-token prediction on massive corpora, combined with scaling of data, parameters, and compute, could produce broad and transferable capabilities. The GPT family demonstrated this trajectory, and the empirical scaling laws that emerged from this period shaped research investment across the field.

The instruction-following era introduced supervised fine-tuning and preference optimisation, including reinforcement learning from human feedback, to align base models with human instructions. The model's task shifted from text continuation to goal-directed response generation, enabling deployment as a general-purpose assistant.

The reasoning era introduced inference-time compute scaling: models trained with verifiable rewards in mathematics, code, and formal settings learned to allocate additional computation to difficult problems, producing longer and more deliberate reasoning chains before committing to an answer.

The agentic era optimises models to call tools reliably, maintain coherent state across long contexts, and operate within loops that incorporate external feedback from tool results and environment observations. This is the setting for which the rest of this book provides architectural guidance.

This progression explains why agentic AI is a continuation of the language modelling research programme rather than a departure from it. Agents are the systems layer built on top of the modelling stack; their capabilities and limitations are directly shaped by where the underlying model sits on this trajectory.

3.5 Summary

Frontier models are the capability substrate of agentic systems. The original Transformer introduced attention as a content-dependent communication mechanism; modern frontier decoders extend that foundation with long-context positional representations, efficient attention variants, normalisation improvements, sparse mixture-of-experts layers, post-training alignment, reasoning optimisation, and multimodal input handling. Model selection should be grounded in a capability profile evaluated inside the actual agent workflow, with benchmark results interpreted as system-level measurements rather than intrinsic model properties.

Chapter 4

Cognitive Architecture

The intelligence layer requires a cognitive architecture which defines how the model makes decisions over time.

4.1 The Agent Loop

The defining property of an agentic system is that it pursues a goal across multiple steps rather than producing a single response. This iterative structure is what the cognitive architecture governs, and it is what distinguishes an agent from a prompted language model.

The loop is necessary because the tasks assigned to agents are rarely solvable in a single pass. A software engineering task may require reading failing tests, forming a reasoning trace about the probable cause, modifying code, running tests again, observing a new failure, revising the reasoning, and repeating. A research task may require querying multiple sources, discovering that initial results are insufficient, broadening the search strategy, and synthesising across an expanded set of evidence. In each case the agent must maintain a coherent picture of where it is in the task, what it has learned, and what it should attempt next.

The standard formulation of this loop, introduced by the ReAct framework (Yao et al., 2022), interleaves reasoning and acting in three repeating stages. In the thought stage, the model generates an internal reasoning trace: it analyses the current state of the task and determines what action is most likely to advance it. In the action stage, it executes that action by calling a tool, retrieving context, producing a partial answer, or requesting clarification. In the observation stage, it receives the result of the action and incorporates it into its reasoning before the next thought. This thought-action-observation cycle continues until a termination condition is met. The critical property of the loop is that the agent learns from each observation before deciding on the next step: a tool call that returns an error is an observation that informs the subsequent thought, rather than a terminal failure.

For the loop to function across multiple iterations, the agent must maintain state. State is the accumulated record of what the agent has done and learned: the user's original input, the message history, retrieved documents, tool outputs, the current plan, completed steps, unresolved questions, and accumulated cost.

4.2 Clarification and Autonomy

A recurring decision in the agent loop is whether to seek clarification from the user or to proceed autonomously. The appropriate choice depends on the risk profile of the next action and the

recoverability of an error.

Clarification is warranted when required parameters are absent, when multiple plausible interpretations of the user's intent would lead to materially different actions, when the next action is irreversible, when the cost of an incorrect action is high, or when the applicable permissions are ambiguous. Autonomous continuation is warranted when the next step carries low risk, when the agent can verify the result of its action before committing to it, when the operation is reversible, when the user has explicitly delegated the task, or when missing information can be retrieved safely without user involvement. The design of this decision boundary is a governance choice as much as a technical one: it determines the degree of autonomy the agent exercises and the degree of accountability it shares with the user.

4.3 Stopping Conditions

Because the loop has no natural end when the task is open-ended, the cognitive architecture must specify explicit termination conditions. Without them the agent continues iterating past the point at which the task is complete, consuming budget and potentially degrading a solution that was already correct. Loop termination is itself a model judgment, rather than a deterministic rule. The agent uses the language model to assess whether the current state of the task satisfies the original goal. This means the stopping decision is subject to the same reasoning capabilities and failure modes as any other step in the loop: the model may judge a task complete when it is not, or continue iterating when sufficient evidence already exists.

4.4 Summary

The cognitive architecture controls how the model reasons and acts over time. It structures the agent's execution as a thought-action-observation loop, allocates deliberation budgets, maintains a clear separation between observations and the reasoning trace, governs the boundary between clarification and autonomous action, and enforces termination conditions. The prompt and skill content that informs each reasoning step belongs to the memory and knowledge layer; the cognitive architecture determines how and when that content is used, and for how long the agent continues to pursue the goal.

Part II

The Action Layer

Chapter 5

The Action Layer

The action layer is the boundary at which model decisions become effects on external systems. A model selects an action; the action layer executes it, validates it, records the result, and returns an observation. The design of this layer determines not only what the agent can do, but how reliably and safely it does it.

A common but misleading simplification is to classify tools as deterministic. The relevant distinction is between tools whose outputs are reproducible and whose side effects are predictable, and those where neither holds. A schema validator is fully reproducible. A database lookup is reproducible for a fixed state. A web search varies because rankings change, pages are updated, and snippets are re-extracted. A sub-agent wrapping another model introduces its own reasoning loop and may reach a different conclusion on each invocation. The architecture must treat each tool according to its actual reproducibility and side-effect profile.

5.1 Schemas as Behavioural Contracts

A tool schema is the primary mechanism by which the model learns what a tool does, when it is appropriate to call it, and what constitutes a valid invocation. A well-designed schema gives the tool an unambiguous name, a description that specifies both the conditions under which the tool should be used and those under which it should not, typed arguments with explicit constraints, and a structured output format.

Schema quality has a direct and measurable effect on tool selection fidelity. When two tools have overlapping descriptions, the model resolves the ambiguity arbitrarily. When argument names are vague, parameters are filled incorrectly. When output fields are verbose or unstructured, the model may extract the wrong field or ignore the result entirely. Research has confirmed that poor schema design is a primary driver of tool hallucination, cases where the model selects the wrong tool, supplies malformed parameters, or bypasses the tool entirely and fabricates an output that mimics what the tool would have returned. The schema is therefore a behavioural control: its precision determines the quality of the agent's decisions at the action boundary.

The following example illustrates a well-formed tool definition using the Strands decorator pattern used later in the implementation chapters. The name is unambiguous, the description specifies both the purpose and the condition for use, arguments are typed with explicit constraints, and the return type is structured:

```
from strands import tool
from pydantic import BaseModel, Field
```

```
class JobStatusResult(BaseModel):
    job_id: str
    status: str # "pending" | "running" | "completed" | "failed"
    progress_pct: float = Field(ge=0.0, le=100.0)
    error_message: str | None = None
    summary: str

@tool
def get_job_status(job_id: str) -> JobStatusResult:
    """
    Retrieve the current execution status of a processing job.
    Use this tool after triggering a job with run_job, and before
    retrieving results with get_results. Do not call this tool
    if the job_id is unknown or has not yet been submitted.
    """
    raw = _fetch_job_status(job_id)
    return JobStatusResult(
        job_id=job_id,
        status=raw["status"],
        progress_pct=raw.get("progress", 0.0),
        error_message=raw.get("error"),
        summary=(
            f"Job {job_id} is {raw['status']} "
            f"({raw.get('progress', 0):.0f}% complete)."
        ),
    )
```

Several design choices are worth making explicit. The description instructs the model both when to call the tool and when to refrain. This negative condition is as important as the positive one because it reduces spurious invocations. The return type is a typed Pydantic model rather than a raw dictionary, which enforces the output contract at the Python level and makes the schema visible to the framework. The `summary` field gives the model a pre-formatted string it can incorporate directly into a response without having to synthesise one from the structured fields.

5.2 Read Tools and Write Tools

Tools differ in their side effects, and that difference governs the level of autonomy the agent is granted. Read tools, including search, retrieval, inspection, calculation, carry no persistent consequences and are appropriate for autonomous invocation. Write tools, those that modify external state, send communications, or trigger processes, might require explicit authorisation in the schema and, for consequential operations, a human approval step before execution.

5.3 Agent-as-Tool and the Opacity Problem

When a subtask is sufficiently complex to warrant its own reasoning loop, such as a research task that requires searching multiple sources, a code review that requires inspecting a patch against a set of criteria, a planning task that decomposes a broad request into ordered steps, it can be encapsulated as a sub-agent and exposed to the parent agent as a tool. The parent agent supplies an input and receives an observation; the internal execution is hidden.

This pattern introduces a structural opacity problem. The parent agent cannot inspect the

sub-agent's intermediate decisions unless the sub-agent exposes them through a shared tracing infrastructure. Research on multi-agent failure modes has found that coordination breakdowns, failures at precisely this boundary, represent the largest single category of failure in production multi-agent systems. The consequence is that an agent-as-tool pattern requires nested traces: the sub-agent's execution must appear as a child span within the parent trace, with its tool calls, intermediate reasoning, and output all visible at the same level of detail as any other tool invocation. The diagnostic value of the trace collapses at the delegation boundary when this structure is absent.

5.4 Summary

The action layer exposes external capabilities through typed invocation boundaries. Each tool is a contract: its schema governs how the model selects and invokes it, its read or write classification governs how much autonomy is appropriate, and its output design governs how well the model can reason from the result. When a tool is itself an agent, the contract extends to require that the sub-agent's execution be fully traceable from the parent context. The evaluation chapter builds directly on this structure: scores attached to trajectory spans measure precisely whether each tool was selected correctly, invoked with valid arguments, and its result incorporated appropriately into subsequent reasoning.

Chapter 6

Tools, Function Calling, and MCP

The Model Context Protocol (MCP) is a standardised interface for exposing tools, prompts, and resources to model hosts. It makes the action layer portable: tools can be implemented once and reused across agents, development environments, desktop applications, and cloud deployments.

6.1 The Function-Calling Loop

When a model is given access to tools, each reasoning step follows a structured cycle. The agent framework presents the model with the user request alongside the available tool schemas. The model determines whether the current state of the task requires a tool invocation, and if so emits a structured call containing the tool name and a JSON argument object. The framework validates that call against the schema before passing it to the tool. The tool runs, and the framework returns its output to the model as an observation. The model then either synthesises a final answer or initiates another step.

The model does not require knowledge of the tool's implementation. It requires a precise external contract. This is why schema design, as discussed in the previous chapter, often has greater influence on agent behaviour than prompt prose: a well-specified schema reduces ambiguity before the model reasons, whereas a prompt correction must override a confusion the model has already formed.

6.2 MCP Primitives

The MCP specification defines three principal abstractions.[1] Tools are model-invoked capabilities that interact with external systems, APIs, computations, or stateful actions; each tool is exposed by a server with a name, metadata, and an input schema.[2] Resources are contextual data items exposed by a server and selected by the application or client; a resource may be a file, a document, a database record, or a repository item. Prompts are reusable templates and workflow instructions that clients can discover and retrieve from a server.[3]

This distinction maps directly onto the layered architecture of this book. Tools primarily belong to the action layer. Resources and prompts primarily support the memory and knowledge layer. All three require security boundaries: because any client that can reach an MCP server can invoke

[1] MCP specification: https://modelcontextprotocol.io/specification/2025-06-18.

[2] MCP tools specification: https://modelcontextprotocol.io/specification/2025-06-18/server/tools.

[3] MCP prompts specification: https://modelcontextprotocol.io/specification/2025-06-18/server/prompts.

its tools, the server must enforce authentication and authorisation independently of the consuming agent.

6.3 Local Tools and Remote Tools

Tools can run inside the agent process or behind a remote endpoint. A local tool is a plain function co-located with the agent code; it is straightforward to prototype and debug, but its availability is limited to the agent that defines it and its security controls are scoped to the process. A remote tool runs behind an API, a serverless function, an MCP server, or a managed gateway; it introduces network overhead but enables the tool to be shared across multiple agents, governed through identity and policy controls, audited centrally, and scaled independently of the agent.

The practical progression is to develop and stabilise a tool locally, establish its schema and write tests against it, and then promote it to a remote endpoint when it must be shared or brought under organisational governance.

6.4 Validation Before Execution

Tool call validation occurs in two distinct places, and conflating them is a common source of production failures. The first is structural validation: the agent framework checks that the model's emitted arguments conform to the tool's schema before the tool is invoked. When a tool's input type is declared precisely, using typed function signatures and constrained field definitions, the framework can enforce this contract automatically and fail early if the model's output does not conform. The second is authorisation: an independent component verifies that the caller's identity is permitted to invoke the requested capability. This separation ensures that schema conformance and access control are enforced by distinct mechanisms, neither of which depends on the model's own judgment.

6.5 Tool Hallucination

Tool hallucination occurs when the model invokes the wrong tool, constructs arguments that the schema does not support, or reasons as though a tool returned information it did not return. Schema precision is the primary structural mitigation, as established in the previous chapter. At the observability level, the agent's user interface should surface which tools were called and with what arguments, making the model's action choices visible to the user rather than concealing them behind a final answer. At the output level, the model can be instructed to include a structured attestation field in its response, for example, a flag indicating that a consequential action such as submitting a payment was executed, which creates an explicit, checkable link between the model's stated behaviour and the tool invocations recorded in the trace. These mitigations are complementary: schema design reduces the rate of hallucination, observability surfaces instances that occur, and structured attestation makes them detectable in automated evaluation.

6.6 Summary

MCP standardises the tool interface so capabilities are portable across agents and deployment environments. Reliable tool use depends on precise schemas, framework-level validation that is independent of the model, and observability infrastructure that makes every invocation inspectable.

The distinction between local and remote tools is a governance boundary as much as an architectural one: the point at which a tool moves to a remote endpoint is the point at which organisational identity, policy, and audit controls become enforceable.

Part III

The Memory and Knowledge Layer

Chapter 7

The Memory and Knowledge Layer

The memory and knowledge layer determines what information is available to the model at the moment of decision. Its function is to construct the context window with the subset of available information that is most relevant and most likely to support a correct and grounded response. Context engineering is the discipline of designing, assembling, and maintaining the information environment available to an agent at inference time, curating the optimal set of tokens during inference, including information beyond the prompt itself.

7.1 Why Memory and Knowledge Form a Single Layer

Memory and knowledge are distinct in origin but unified in function. Memory accumulates through interaction: it records user preferences, past decisions, prior conversation turns, and outcomes. Knowledge is sourced externally: documents, policies, codebases, manuals and databases. Both must pass through the same context engineering process before reaching the model's context window. Context engineering governs this selection, deciding which pieces of memory and which fragments of knowledge to include for a given task, in what form, and at what level of compression.

The two are treated as a single layer because failures in either produce the same downstream effect on grounding: the model reasons from an incomplete or misleading context window. An agent that retrieves irrelevant document chunks, omits a critical policy, or fails to surface a previously stated user preference will produce ungrounded outputs regardless of the model's underlying capability.

7.2 Types of Memory

Memory in agentic systems is usefully classified by its temporal scope and function. Working memory is the immediate task context: the current messages, active plan, tool outputs, and unresolved constraints that the agent is reasoning over in the present turn. Short-term memory spans a session: it preserves conversational history across turns and enables the agent to maintain coherence without re-establishing context on every exchange. Episodic memory records past events across sessions: what the user requested, what the agent did, and what outcome resulted. Semantic memory holds stable extracted facts: user role, project names, known issues, and domain definitions that remain valid across interactions. Preference memory captures standing instructions about tone, format, default tools, and preferred workflows. Procedural memory contains reusable packages of know-how: skills, checklists, playbooks, and workflow templates that the agent can retrieve and apply when a task matches their scope.

7.3 Skills as Procedural Knowledge

A skill occupies a specific position in this taxonomy. It is neither a model weight nor a tool: it is a retrievable unit of procedural knowledge that the system injects into the context window when the current task requires it. A skill may take the form of a domain-specific instruction fragment, a step-by-step checklist for a recurring workflow, a repository development guide, a troubleshooting playbook, or a bundle of tool-use rules paired with expected output formats.

The key property of a skill is that it is selected, versioned, and evaluated independently of the model. A skill whose content is outdated introduces incorrect procedural guidance, and one that is irrelevant to the current task adds noise to the context window and degrades grounding. Skills therefore belong in the knowledge layer and are subject to the same retrieval, quality, and freshness requirements as any other document.

A common representation is a structured Markdown document that pairs machine-readable frontmatter with human-readable procedural content. The frontmatter declares the skill's name, its intended scope, and the tools it is permitted to invoke; the body encodes the step-by-step procedure the agent should follow when the skill is active:

```
---
name: data-pipeline-diagnosis
description: >
  Procedure for diagnosing failures in a data processing pipeline.
  Use when the user reports missing, delayed, or malformed output.
allowed-tools:
  - fetch_job_logs
  - get_job_status
  - inspect_schema
---

## Diagnosis Flow

1. Retrieve the job identifier from the user or from recent job history.
2. Call get_job_status to determine whether the job completed,
   failed, or is still running.
3. If the job failed, call fetch_job_logs and identify the first
   error entry.
4. Call inspect_schema to verify that the input data conforms to
   the expected format.
5. Summarise the root cause and propose a remediation step.
```

This format makes the skill inspectable, diffable, and testable. A new version of the skill can be evaluated against a dataset of past failures before it replaces the previous version in the retrieval index. The agent framework retrieves the skill document when the current task matches its declared scope and injects its content into the context window alongside the user's request.

7.4 Context Compression

Context windows are finite and carry a cost proportional to their length. Compression is the process of reducing the volume of context while preserving the information most relevant to the task at hand. Common techniques include summarising prior conversation turns through memory consolidation, retaining only the highest-ranked retrieval chunks, and converting verbose tool outputs into compact structured fields containing only the decision-relevant values.

Compression introduces its own failure modes. A consolidation step may omit a critical qualification present in the original. A memory extraction step may store a temporary instruction as a permanent preference. A retrieval filter may exclude the document that contains the correct grounding evidence. For this reason, memory and context transformations require the same evaluation discipline as model outputs: they are active interventions in the context engineering pipeline that can introduce errors at any step.

7.5 Memory Write Policy

Selective retention is a design requirement. An agent that stores every piece of information it encounters produces a memory that degrades in utility as it accumulates noise, outdated facts, and low-confidence inferences. A principled write policy stores information when the user has explicitly requested retention, when the fact is stable and reusable across future sessions, when storage improves personalisation or session continuity in a way the user has consented to, and when the information originates from a trusted source. Sensitive information that serves no persistent purpose, temporary context that applies only to the current task, and content inferred with low confidence warrant exclusion from long-term memory.

The system should expose memory to inspection and correction. A user who can query what the agent has retained and modify or delete individual entries has meaningful control over the grounding context the agent will assemble in future sessions.

7.6 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) is the standard pattern for grounding model responses in external knowledge. It proceeds through four operations. Chunking divides source documents into retrieval units of appropriate granularity, each carrying metadata that identifies its origin, recency, and scope. Embedding maps each chunk into a vector representation that supports semantic similarity search. Retrieval selects the candidate chunks most relevant to the current query, typically by combining vector similarity with keyword or metadata filters. Context assembly arranges the selected chunks into the context window in a form the model can reason from, managing ordering, deduplication, and length constraints.

Each of these operations is a potential source of grounding failure and should be tested independently. A chunking strategy that splits sentences across unit boundaries degrades retrieval precision. An embedding model that has drifted from the domain of the documents produces unreliable similarity scores. A retrieval step that returns the highest-scoring chunks without diversity may miss the document that contains the correct answer. A context assembly step that places the most relevant content at the end of a long context window may cause the model to underweight it due to the well-documented lost-in-the-middle effect. Treating RAG as a pipeline of testable components rather than a monolithic retrieval call is what makes its failure modes diagnosable.

7.7 Summary

The memory and knowledge layer is where skills, context engineering, long-term memory, and retrieval infrastructure belong. Its function is to construct the context window with the right information at the right time. Production agent design is therefore as much a context engineering discipline as a model engineering one: the quality of the context assembled for each task is as consequential as the capability of the model that receives it.

Part IV

The Security and Governance Layer

Chapter 8

The Security and Governance Layer

Agents introduce a category of risk that conversational systems do not. A model that produces an incorrect response misleads a user. An agent that issues an incorrect instruction to a tool can leak sensitive data, send communications, modify records, or deploy broken code to production. Security and governance are therefore foundational properties of the architecture, rather than capabilities added after the core system is built.

8.1 The Attack Surface

The concept of an attack surface describes the set of entry points through which an adversary can influence a system's behaviour. In agentic systems, this surface is substantially wider than in conventional applications. The model's context window is an attack vector: every piece of content that enters it, including retrieved documents, tool outputs, emails, web pages, tickets, and code, can contain instructions that attempt to redirect the model's behaviour. The agent's memory is a further attack vector: content written into long-term memory during one session can influence the model's reasoning in future sessions, a class of attack known as memory poisoning. The tools themselves constitute a third vector: an agent with access to credentials, external APIs, or write operations over persistent storage can be induced to exercise those capabilities in ways that were not authorised.

The principle of least privilege is the foundational response to this expanded attack surface. Each component of the system, including the agent, each tool, and each memory store, should hold only the permissions required to perform its declared function, scoped to the current task, and enforced by the infrastructure rather than assumed from the model's compliance with its system prompt. In agentic contexts, least privilege means limiting an agent's access to only the resources and actions explicitly required to complete its current task, not its general purpose, its potential future tasks, or everything it might conceivably need.

8.2 Identity, Authentication, and Authorisation

Every request in an agentic system should be associated with an identity. That identity may belong to a human user, a service account, a tenant, an organisation, an agent, or a tool. In multi-agent systems, the agent itself carries an identity so that downstream tools can distinguish between a direct human request and an automated action, a distinction that matters for audit, for permission scoping, and for accountability.

Authentication and authorisation operate at two boundaries. Inbound authentication verifies the user or client calling the agent. Outbound authorisation verifies, before any tool executes, whether the authenticated identity holds the permission required to invoke it. When a tool must call a downstream service, the credentials for that call should be scoped, rotated, and injected into the tool execution environment by a secrets manager rather than passed through the model. The model may know that a user holds a particular role; it should have no visibility into OAuth tokens, API keys, or database connection strings.

Permissions should be scoped along multiple axes: user or tenant, data source, tool, action type, environment, time window, budget, and approval requirement. A deployment agent, for example, may be permitted to run tests in any branch, deploy to a staging environment after automated checks pass, and deploy to production only after an authorised engineer has approved the change. Each axis of scoping reduces the consequence of a compromised or misdirected action.

8.3 Prompt Injection

Prompt injection occurs when content from an untrusted source attempts to redirect the model's behaviour. Direct injection arrives in the user's own message. Indirect injection is embedded in content the agent retrieves or processes, such as a web page, a PDF, an email body, a repository file, or a tool output, and is particularly consequential because the user may have no visibility into the injected instruction.

The primary structural mitigations are architectural. Context should be labelled by source and authority level so the model can distinguish between a user instruction and a retrieved passage. Tool access should be scoped to the authenticated user's identity so that an injected instruction cannot invoke a capability the user does not hold. Tool calls should be validated by an independent component rather than trusted on the basis of the model's output alone. Data flows from sensitive sources to external sinks should be explicitly governed. Write operations should require approval for consequential actions. High-risk tools should operate against an allowlist. Every consequential action should be logged with sufficient context to reconstruct the decision that produced it. Infrastructure enforcement is what gives these controls their guarantee.

8.4 Guardrails

Guardrails are validation controls placed at specific points in the agent's execution pipeline, enforcing boundaries that the model itself cannot reliably enforce. An input guardrail inspects the user's request before it reaches the model, blocking unsafe or unsupported inputs. A retrieval guardrail restricts which documents are accessible during context assembly, for example by applying tenant-specific index filters. A tool guardrail validates a tool call before execution, blocking actions such as sending an external communication that contains sensitive data. An output guardrail inspects the model's final response, redacting personal data or flagging policy violations before the response reaches the user. A policy guardrail enforces business rules such as requiring human approval above a cost threshold. A runtime guardrail constrains the execution environment itself, sandboxing shell access and network calls to prevent lateral movement.

The common property across all guardrail types is that they operate independently of the model. The model may propose an action; the guardrail determines whether it executes.

8.5 Data Flow Control

A governed agent architecture tracks the provenance of information from source to sink. A source is where information enters the agent's context: a user profile, a private document, an email, a code repository, a database record, or a web page. A sink is where information leaves the system: a final response, an outbound email, a message, a ticket update, a file write, or an external API call.

Data exfiltration occurs when content from a protected source reaches a sink the user or organisation has not authorised. An agent with read access to internal documents and write access to external communication channels must be governed by explicit data flow rules that an injected instruction cannot override. These rules belong in the infrastructure layer, enforced at the tool boundary.

8.6 Human Approval

For actions that are irreversible or consequential, such as modifying or deleting records, deploying code, submitting a financial or legal document, or sharing internal data outside the organisation, a human approval step is one available control. The appropriate level of human oversight depends on the risk profile of the action and the degree of automation the use case requires. A fully automated pipeline may accept higher risk in exchange for throughput; a governed enterprise deployment may route every write action through an approval queue. The design choice is between automation and accountability, and the right point on that spectrum is determined by the consequences of an unchecked error.

Approval events carry evaluation value regardless of where that point is set. A rejected approval records a case where the agent's proposed action diverged from human judgment. Collected systematically, rejected approvals become labelled examples for the evaluation dataset and regression tests for subsequent agent versions.

8.7 Sandboxing

A sandbox is an environment that replicates production conditions without connecting to production systems. In a staged deployment pipeline, the sandbox stage, typically referred to as gamma, is where the agent is tested against realistic data and realistic tool schemas before any change is promoted to production. The CI/CD pipeline should include a sandbox stack where prompt changes, tool schema updates, and new agent capabilities can be exercised and evaluated in isolation. Capabilities validated in the sandbox environment carry a reproducible evidence record; those that have not been validated there should remain outside the production deployment.

8.8 Governance and Auditability

Governance requires that the organisation can reconstruct, for any past interaction, who invoked the agent, what the model decided, which tools were called, and which policy checks ran. Observability infrastructure that captures every model call, tool invocation, and policy check as a named span with associated metadata is the prerequisite for this. Agent versioning ensures that a trace can be associated with the exact prompt, tool schemas, and skill documents that were active at the time of the interaction, making it possible to reproduce and evaluate historical behaviour as the system evolves.

8.9 Summary

The security and governance layer enforces the boundaries within which the agent operates. The attack surface spans the context window, the memory layer, and the tool interfaces; the principle of least privilege is the foundational response to each. Identity, authentication, and authorisation establish who may act and on what. Guardrails enforce boundaries at every stage of the execution pipeline independently of the model. Prompt injection, data flow control, human approval calibrated to the risk profile of each action, sandboxed validation before promotion, and structured auditability are the core concerns of this layer. Each of them is an infrastructure property enforced independently of the model, and each connects directly to the evaluation and deployment practices covered in subsequent chapters.

Part V

Applications: Building Agents in Production

Chapter 9

Building Agents: Strands and LangGraph

The previous chapters established the conceptual architecture of an agent: layers of intelligence, action, memory, and security. This chapter makes those abstractions concrete by implementing the same assistant twice, once with Strands and once with LangGraph. The tools, the gateway, the skills, and the memory resource are identical in both cases. Only the way the agent loop is assembled differs. Examining both implementations in parallel surfaces what each framework provides and what it leaves to the developer.

The assistant connects to backend capabilities through an MCP gateway secured with AWS Signature Version 4 (SigV4). A `SKILL.md` document encodes the operating procedure. AgentCore Memory keeps track of conversational state and user preferences across sessions. The book follows the order in which components are typically written: tools first, then the agent loop, then memory, then the entrypoint.

9.1 Step 1: Define the tools

Tools are the agent's interface to the world. Before picking a framework, define what the agent can do. Built-in tools are plain Python functions decorated with the `@tool` decorator. The Strands import and the LangGraph import use different packages, but the pattern is identical: a function, a docstring, and an optional return type.

```
# Strands import
from strands import tool

# LangGraph import
from langchain_core.tools import tool

@tool
def get_current_time() -> dict[str, str]:
    """Return the current UTC time."""
    from datetime import datetime, timezone
    return {"utc": datetime.now(timezone.utc).isoformat()}

@tool
def echo(message: str) -> dict[str, str]:
    """Return the message back to the caller. Useful for testing."""
```

```
    return {"response": message}
```

These two tools illustrate the pattern: a decorated function, a docstring that the model uses to decide when to call the tool, and a typed return value. In practice, built-in tools handle lightweight operations that do not require a gateway round-trip, such as simple lookups, formatting, or diagnostic checks. The more capable tools are loaded dynamically from the AgentCore Gateway at startup, as described in the next section.

9.2 Step 2: Load gateway tools over MCP

Gateway tools arrive via the MCP protocol. The two frameworks use different MCP clients, so the gateway loading code diverges here. Strands uses `MCPClient` from the `strands.tools.mcp` package and loads tools synchronously during initialisation. LangGraph uses `MultiServerMCPClient` from `langchain-mcp-adapters` and loads tools asynchronously.

Strands gateway loader.

```
import os
import atexit
from mcp.client.streamable_http import streamablehttp_client
from strands.tools.mcp import MCPClient

def _gateway_urls() -> list[str]:
    raw = os.environ.get("GATEWAY_URL", "")
    return [url.strip() for url in raw.split(",") if url.strip()]

def _make_client(url: str) -> MCPClient:
    region = os.environ.get("GATEWAY_REGION", "us-west-2")
    auth = SigV4HTTPAuth(service=SERVICE_NAME, region=region)
    return MCPClient(
        lambda: streamablehttp_client(
            url=url,
            headers={
                "Accept": "application/json, text/event-stream"
            },
            auth=auth,
        )
    )

class GatewayToolPool:
    def __init__(self) -> None:
        self.clients: list[MCPClient] = []
        self.tools_list: list = []
        self.load_tools()
        atexit.register(self.stop)

    def load_tools(self) -> None:
        urls = _gateway_urls()
        clients = []
        for url in urls:
            client = _make_client(url)
            client.__enter__()
            clients.append(client)
```

```
        tools = []
        for client in clients:
            tools.extend(client.list_tools_sync())
        self.clients = clients
        self.tools_list = tools

    def tools(self) -> list:
        return list(self.tools_list)

    def stop(self) -> None:
        for client in self.clients:
            try:
                client.__exit__(None, None, None)
            except Exception:
                pass
```

`GatewayToolPool` enters each client's context manager on initialisation and registers a cleanup function with `atexit`. The Strands agent holds a reference to this pool and calls `tools()` to get the current list. Because Strands holds persistent MCP client connections, the pool must be kept alive for the duration of the container's lifetime.

LangGraph gateway loader.

```
import os
from langchain_mcp_adapters.client import MultiServerMCPClient

async def load_gateway_tools() -> list:
    urls = _gateway_urls()
    if not urls:
        return []

    region = os.environ.get("GATEWAY_REGION", "us-west-2")
    client_config = {}
    for index, url in enumerate(urls):
        client_config[f"backend_{index}"] = {
            "transport": "http",
            "url": url,
            "auth": SigV4HTTPAuth(
                service=SERVICE_NAME, region=region
            ),
            "headers": {
                "Accept": "application/json, text/event-stream"
            },
        }

    client = MultiServerMCPClient(client_config)
    return await client.get_tools()
```

The LangGraph loader is a coroutine. It awaits `get_tools()` and returns LangChain-compatible tool objects that can be passed directly to `create_react_agent`. Unlike the Strands pool, `MultiServerMCPClient` is stateless by default: each tool invocation creates a fresh MCP session, executes the tool, and closes the session. This means the LangGraph implementation does not require session affinity at the container level, but it incurs a small per-call connection overhead.

9.3 Step 3: Encode operating procedures in SKILL.md

Tools tell the agent *how* to call a capability. A skill tells the agent *when* and *why* to use capabilities in a particular order. The assistant has one skill: a Markdown document that defines the standard operating procedure for the tasks it is responsible for.

The skill document starts with YAML frontmatter that names the skill and declares which gateway tools it is allowed to use:

```
---
name: data-processing-assistant
description: >
  Standard operating procedure for data retrieval,
  job execution, and result review.
allowed-tools:
  - backend___fetch_data
  - backend___run_job
  - backend___check_status
  - backend___get_results
  - backend___create_override
---

## Processing Flow

1. Confirm the input identifier with the user.
2. Fetch the relevant data with fetch_data.
3. Trigger the processing job with run_job.
4. Poll completion status with check_status.
5. Retrieve and summarise results with get_results.
```

The skill registry reads these files and returns metadata for the system prompt and full content for the agent's operating context:

```
from pathlib import Path
from typing import Any

def skill_files(base_dir: Path | None = None) -> list[Path]:
    root = base_dir or Path(__file__).parent
    return sorted((root / "skills").glob("*/SKILL.md"))

def parse_skill_frontmatter(
    skill_file: Path,
    base_dir: Path | None = None,
) -> dict[str, Any]:
    text = skill_file.read_text(encoding="utf-8")
    root = base_dir or Path(__file__).parent
    metadata: dict[str, Any] = {
        "name": skill_file.parent.name,
        "description": "",
        "allowedTools": [],
        "path": str(skill_file.relative_to(root)),
    }
    if not text.startswith("---"):
        return metadata
    end = text.find("\n---", 3)
```

```
    if end == -1:
        return metadata
    current_key = ""
    for raw_line in text[3:end].splitlines():
        line = raw_line.rstrip()
        if not line.strip():
            continue
        if (line.startswith(" - ")
                and current_key == "allowed-tools"):
            metadata["allowedTools"].append(line[4:].strip())
            continue
        if ":" in line and not line.startswith(" "):
            key, value = line.split(":", 1)
            current_key = key.strip()
            if current_key == "name":
                metadata["name"] = value.strip()
            elif current_key == "description":
                metadata["description"] = value.strip()
            elif current_key == "allowed-tools":
                metadata["allowedTools"] = []
    return metadata

def loaded_skills(base_dir: Path | None = None) -> list[dict]:
    root = base_dir or Path(__file__).parent
    return [
        parse_skill_frontmatter(f, root)
        for f in skill_files(root)
    ]
```

The frontmatter parser keeps the logic minimal: it reads YAML keys without a YAML library so the agent package stays lean. Strands consumes the skills directory through a plugin. LangGraph reads the full document content and embeds it in the system prompt.

9.4 Step 4: Build the Strands agent

Strands takes a declarative approach. The developer provides a model, a session manager, plugins, tools, and a system prompt. The framework owns the agent loop: it decides when to call the model, when to invoke tools, and how to stream results back. The agent's code describes *what* the agent is, not *how* its loop runs.

```
import json, os
from pathlib import Path
from strands import Agent, AgentSkills
from bedrock_agentcore.memory.integrations.strands\
    .session_manager import AgentCoreMemorySessionManager

def _model_id() -> str:
    return os.environ.get(
        "MODEL_ID", "us.amazon.nova-2-lite-v1:0"
    )

_agent: Agent | None = None
_gateway_pool: GatewayToolPool | None = None
```

```
def _gateway_tools() -> list:
    global _gateway_pool
    if _gateway_pool is None:
        _gateway_pool = GatewayToolPool()
    tools = _gateway_pool.tools()
    if not tools:
        raise RuntimeError("No gateway tools loaded")
    return tools

def _skills_plugin():
    return AgentSkills(
        skills=str(Path(__file__).parent / "skills")
    )

def _memory_session_manager(
    runtime_session_id: str,
) -> AgentCoreMemorySessionManager:
    actor_id, session_id = _parse_runtime_session_id(
        runtime_session_id
    )
    # Replace with your AgentCore Memory ID and region
    return AgentCoreMemorySessionManager(
        agentcore_memory_config={
            "memory_id": os.environ["MEMORY_ID"],
            "actor_id": actor_id,
            "session_id": session_id,
        },
        region_name=os.environ.get(
            "MEMORY_REGION", "us-west-2"
        ),
    )

def _agent_instance(runtime_session_id: str) -> Agent:
    global _agent
    if _agent is None:
        gateway_tools = _gateway_tools()
        _agent = Agent(
            model=_model_id(),
            session_manager=_memory_session_manager(
                runtime_session_id
            ),
            plugins=[_skills_plugin()],
            tools=[
                get_current_time,
                echo,
                *gateway_tools,
            ],
            system_prompt=(
                "You are a helpful assistant. "
                "Use the available gateway tools to complete "
                "tasks. Follow the loaded skills for standard "
                "operating procedures.\n\n"
                f"Loaded skills: "
```

```
            f"{json.dumps(loaded_skills())}\n\n"
        ),
        callback_handler=None,
    )
    return _agent
```

Three things are worth observing here.

First, memory is attached as a *session manager*. The Strands `Agent` receives an `AgentCoreMemorySessionManag` and from that point treats memory as a framework service. It retrieves relevant history, maintains conversational continuity, and writes new interactions to memory as part of the normal agent loop. The application code does not manage memory explicitly.

Second, skills arrive through the `AgentSkills` plugin. The plugin scans the `skills/` directory, parses each `SKILL.md`, and makes its procedures available to the agent without the developer having to embed content manually in a prompt.

Third, the container is pinned to a single session. If the same container receives a request for a different session identifier, the agent raises an error. This is a deliberate trade-off: because Strands holds stateful MCP client connections through the `GatewayToolPool`, reusing a container across sessions would corrupt session state. AgentCore Runtime handles this constraint by routing requests to the correct container.

9.5 Step 5: Build the LangGraph agent

LangGraph makes the agent loop explicit. The developer builds a graph with nodes, edges, and hooks. The prebuilt `create_react_agent` supplies a default ReAct loop: model call, tool call, observation, repeat. However, every part of the loop can be extended, replaced, or instrumented.

Memory components. LangGraph splits memory into two objects. The checkpointer stores graph state per thread so conversations can resume after interruption. The store is a semantic store for long-term memory that supports vector search over past interactions.

```
from langgraph_checkpoint_aws import (
    AgentCoreMemorySaver,
    AgentCoreMemoryStore,
)

def _memory_components():
    memory_id = os.environ["MEMORY_ID"]
    region = os.environ.get("MEMORY_REGION", "us-west-2")
    checkpointer = AgentCoreMemorySaver(
        memory_id, region_name=region,
    )
    store = AgentCoreMemoryStore(
        memory_id=memory_id, region_name=region,
    )
    return checkpointer, store
```

The pre-model hook. The pre-model hook runs before every model call. It is the point where the graph reads long-term memory and injects it into the conversation. LangGraph exposes this hook explicitly because the developer may want to log what was retrieved, filter it, or change

the injection strategy for different graph branches. Strands handles equivalent retrieval inside the framework.

```
import uuid, json
from langchain_core.messages import HumanMessage
from langchain_core.runnables import RunnableConfig
from langgraph.store.base import BaseStore

def _pre_model_hook(
    state: dict,
    config: RunnableConfig,
    *,
    store: BaseStore,
) -> dict:
    configurable = config.get("configurable", {})
    actor_id = str(
        configurable.get("actor_id", "default_user")
    )
    thread_id = str(
        configurable.get("thread_id", "default")
    )
    messages = list(state.get("messages", []))

    latest_human = next(
        (
            m for m in reversed(messages)
            if isinstance(m, HumanMessage)
            or getattr(m, "type", None) == "human"
        ),
        None,
    )

    if latest_human is not None:
        store.put(
            (actor_id, thread_id),
            str(uuid.uuid4()),
            {"message": latest_human},
        )
        memories = store.search(
            (actor_id, thread_id),
            query=str(
                getattr(latest_human, "content", "")
            ),
            limit=5,
        )
        if memories:
            memory_lines = [
                json.dumps(
                    getattr(m, "value", m), default=str
                )
                for m in memories
            ]
            messages.insert(
                0,
```

```
                HumanMessage(
                    content=(
                        "Relevant memory context:\n"
                        + "\n".join(memory_lines)
                    )
                ),
            )

    return {"llm_input_messages": messages}
```

The hook saves the latest human message to the store and immediately searches the same namespace for relevant prior context. If any is found, it is prepended to the message list before the model sees the conversation. This is explicit: the developer controls exactly when memory is written, what is searched, and how results are formatted before injection.

Assembling the graph. With tools, memory, and hooks in place, the graph is assembled in a single call:

```
from langchain.chat_models import init_chat_model
from langgraph.prebuilt import create_react_agent

_graph = None
_gateway_tools_cache = None

def _build_system_prompt(gateway_tools: list) -> str:
    tool_names = [
        getattr(t, "name", str(t)) for t in gateway_tools
    ]
    return (
        "You are a helpful assistant. "
        "Use the available gateway tools to complete tasks. "
        "Follow the loaded skills for standard operating "
        "procedures.\n\n"
        f"Available gateway tools: {tool_names}\n\n"
        f"Loaded skills: {json.dumps(loaded_skills())}\n\n"
    )

async def _graph_instance():
    global _graph, _gateway_tools_cache
    if _graph is None:
        _gateway_tools_cache = await load_gateway_tools()
        if not _gateway_tools_cache:
            raise RuntimeError("No gateway tools loaded")

        checkpointer, store = _memory_components()
        region = os.environ.get("AWS_REGION", "us-west-2")
        model = init_chat_model(
            _model_id(),
            model_provider="bedrock_converse",
            region_name=region,
        )
        tools = [
            get_current_time,
            echo,
```

```
            *_gateway_tools_cache,
        ]
        _graph = create_react_agent(
            model=model,
            tools=tools,
            checkpointer=checkpointer,
            store=store,
            prompt=_build_system_prompt(_gateway_tools_cache),
            pre_model_hook=_pre_model_hook,
        )
    return _graph
```

The `create_react_agent` call builds a prebuilt ReAct graph: the model decides which tools to call, each tool is executed, and the observation is fed back to the model until it produces a final answer. Skills in LangGraph are embedded in the system prompt: the skill registry reads each `SKILL.md` document and the `_build_system_prompt` function formats them into a single prompt block. There is no plugin; the developer controls the skill injection point directly.

9.6 Step 6: The entrypoint and session identity

Both implementations wrap the agent in a `BedrockAgentCoreApp` and expose an `@app.entrypoint` function. The entrypoint is the interface between the AgentCore Runtime and the agent's domain logic. Both use the same convention: the `runtimeSessionId` encodes both actor and session using a triple-underscore separator.

```
def _parse_runtime_session_id(raw: str) -> tuple[str, str]:
    if "___" in raw:
        actor_id, session_id = raw.split("___", 1)
        return actor_id.strip(), session_id.strip()
    return "default_user", raw.strip()

def _runtime_session_id_from(payload: dict, context) -> str:
    user_id = str(payload.get("user_id") or "").strip()
    session_id = str(payload.get("session_id") or "").strip()
    if user_id and session_id:
        return f"{user_id}___{session_id}"
    context_session_id = getattr(context, "session_id", None)
    if context_session_id:
        return str(context_session_id)
    raise ValueError(
        "Provide user_id + session_id in the payload, "
        "or invoke through AgentCore with a runtimeSessionId"
    )
```

Strands entrypoint. The Strands entrypoint creates or retrieves the agent instance, then calls `stream_async` with the user prompt. The framework handles tool calling, memory updates, and streaming events internally.

```
from bedrock_agentcore import BedrockAgentCoreApp

app = BedrockAgentCoreApp()
```

```
@app.entrypoint
async def invoke(payload: dict, context=None):
    prompt = str(payload.get("prompt") or "").strip()
    if not prompt:
        raise ValueError("prompt is required")
    if prompt.lower() in {"health", "ping", "status"}:
        yield {"message": {"content": [{"text": "ok"}]}}
        return
    runtime_session_id = _runtime_session_id_from(
        payload, context
    )
    agent = _agent_instance(runtime_session_id)
    async for event in agent.stream_async(prompt):
        yield event
```

LangGraph entrypoint. The LangGraph entrypoint awaits the graph, invokes it with the prompt and session configuration, and yields a single response event extracted from the final messages list.

```
from langchain_core.messages import AIMessage

app = BedrockAgentCoreApp()

def _last_ai_text(messages: list) -> str:
    for message in reversed(messages):
        if (
            isinstance(message, AIMessage)
            or getattr(message, "type", None) == "ai"
        ):
            content = getattr(message, "content", "")
            if isinstance(content, str):
                return content
            if isinstance(content, list):
                return "".join(
                    item.get("text", "")
                    if isinstance(item, dict)
                    else str(item)
                    for item in content
                )
    return ""

@app.entrypoint
async def invoke(payload: dict, context=None):
    prompt = str(payload.get("prompt") or "").strip()
    if not prompt:
        raise ValueError("prompt is required")
    if prompt.lower() in {"health", "ping", "status"}:
        yield {"message": {"content": [{"text": "ok"}]}}
        return
    runtime_session_id = _runtime_session_id_from(
        payload, context
    )
    actor_id, session_id = _parse_runtime_session_id(
```

```
    runtime_session_id
)
graph = await _graph_instance()
result = await graph.ainvoke(
    {"messages": [("human", prompt)]},
    config={
        "configurable": {
            "actor_id": actor_id,
            "thread_id": session_id,
        }
    },
)
text = _last_ai_text(list(result.get("messages", [])))
yield {
    "message": {
        "role": "assistant",
        "content": [{"text": text}],
    }
}
```

The Strands version yields events as they arrive from the framework. The LangGraph version awaits the full graph result and yields one final response. This reflects a deeper difference: Strands is streaming-first by design, while LangGraph's `ainvoke` returns a completed state.

9.7 Step 7: The full picture

With every component defined, the two agents have the same external shape. Both accept a prompt over the AgentCore Runtime boundary. Both authenticate to the same MCP gateway using SigV4. Both load the same skill document. Both store and retrieve memory using the same AgentCore Memory resource. The difference is in what the developer writes.

With Strands, the developer writes an agent *declaration*. The framework owns the loop, the tool dispatch, the memory integration, and the event streaming. The developer chooses model, tools, plugins, and prompt. The control flow is implicit in those choices.

With LangGraph, the developer writes an agent *graph*. The framework provides a graph builder and a default ReAct loop, but the developer can add nodes, attach hooks, add conditional edges, and instrument every transition. Memory integration is explicit: the developer writes the hook that reads memory and decides how to inject it. Skills integration is explicit: the developer reads the skill document and decides how to format it in the prompt.

9.8 Framework comparison

Aspect	**Strands**	**LangGraph**
Agent loop	Owned by the framework; hidden from application code	Explicit graph; developer builds nodes and edges
Gateway auth	`SigV4HTTPAuth` passed to `MCPClient` via `streamablehttp_client`	`SigV4HTTPAuth` passed as `auth` field in `MultiServerMCPClient` config

MCP connection	Persistent connections held in `GatewayToolPool`; tools loaded once at startup	Stateless per tool call; each invocation opens a fresh MCP session
Memory	`AgentCoreMemorySessionManager` as a framework service; retrieval handled internally	`AgentCoreMemorySaver` + `AgentCoreMemoryStore` + explicit pre-model hook
Skills	`AgentSkills` plugin scans the `skills/` directory automatically	Developer reads `SKILL.md` files and embeds content in the system prompt
Streaming	`stream_async()` emits events as the loop runs	`ainvoke()` returns completed state; streaming requires `astream()`
Session affinity	Required: container pinned to a single session because MCP connections are stateful	Not required: stateless MCP sessions allow the container to serve any session
Control flow	Framework decides tool dispatch order	Developer can add conditional edges, interrupts, and custom nodes

9.9 When to choose each framework

The practical question is where complexity lives in the system.

If complexity lives in the tools and operating procedures: the skill is detailed, the gateway exposes many capabilities, the conversation requires careful tool sequencing. Strands keeps the application code compact while the skill documents carry the domain logic. The framework handles the loop; the developer handles the domain.

If complexity lives in the orchestration itself: the agent must coordinate deterministic steps with model-driven steps, expose checkpoints for human approval, branch on domain-specific conditions, or make memory injection visible for evaluation. LangGraph provides explicit structure for representing and testing each part of the trajectory.

Both frameworks integrate with AgentCore Runtime, AgentCore Gateway, and AgentCore Memory. The decision is about where control should live: in the tools and operating procedures, or in the graph itself.

9.10 Summary

Both implementations accept a prompt over the AgentCore Runtime boundary, authenticate to the MCP gateway with SigV4, load tools from the gateway, follow skill-encoded operating procedures, and persist state through AgentCore Memory. The gateway authentication pattern, signing each inbound request with the container's IAM execution role credentials, is the standard approach for any AWS workload calling an IAM-protected service endpoint, and applies equally to both frameworks. What differs is the agent loop. Strands declares an agent and delegates loop control to the framework. LangGraph builds an explicit graph with hooks, memory components, and a system prompt that includes skill content directly. The choice between them is a question of where the developer wants to own control: in the tools and operating procedures, or in the graph itself.

Chapter 10

From local to the cloud: Deploying Agents with AWS AgentCore

Chapter 8 assembled the same assistant in two frameworks, producing two agent implementations that run locally when their environment variables point to real gateway and memory resources. Deployment extends that work into a governed cloud service: it supplies an invocation boundary, an execution role, a memory resource, a governed tool surface, a release mechanism, and the observability needed to diagnose failures in production.

Amazon Bedrock AgentCore provides managed infrastructure for each of these concerns. AWS CDK declares all required resources in code. Examining the CDK stack component by component reveals what each AgentCore service does and why the provisioning order follows the dependencies it does.

10.1 Step 1: Understand what the runtime expects

Before writing infrastructure, it helps to understand the contract the agent code already satisfies. In chapter 8, both agents wrap themselves in a `BedrockAgentCoreApp` and expose exactly one `@app.entrypoint` function:

```
from bedrock_agentcore import BedrockAgentCoreApp

app = BedrockAgentCoreApp()

@app.entrypoint
async def invoke(payload: dict, context=None):
    prompt = str(payload.get("prompt") or "").strip()
    runtime_session_id = _runtime_session_id_from(
        payload, context
    )
    # ... agent logic ...
    yield {
        "message": {
            "role": "assistant",
            "content": [{"text": response_text}],
        }
    }

def main():
```

```
    app.run(host="0.0.0.0", port=8080)
```

`BedrockAgentCoreApp.run()` starts an HTTP server on port 8080. AgentCore Runtime sends invocation requests to this server, supplies the `runtimeSessionId` through the context object, and streams or collects the yielded events. The agent code does not need to know it is running in a container or behind a managed runtime; it only needs to be packaged so that AgentCore can start it.

This separation means the same code can be tested locally by running `python -m agent.app` and passing test payloads directly, and deployed by packaging the `agent/` directory as a Docker image.

10.2 Step 2: Build the tool layer

The runtime does not own the business logic. A well-structured deployment separates tool implementation from agent orchestration by placing domain operations in a Lambda function. The Lambda function is the first resource created in the CDK stack, before the runtime, because the gateway and the runtime both depend on it.

```
import { createToolsResources } from './tools-construct';

const tools = createToolsResources(this, {
  stageName: props.stage.name,
  removalPolicy,
  modelId: DEFAULT_MODEL_ID,
});
```

`createToolsResources` returns a Lambda function and a DynamoDB state table. The Lambda function handles tool invocations from the AgentCore Gateway and any scheduled or event-driven triggers the application requires. The DynamoDB table stores all typed records with a configurable TTL.

This separation gives the deployment a clean ownership model: the agent owns reasoning, the gateway owns tool discovery and protocol, the Lambda owns domain operations, and DynamoDB owns persisted state.

10.3 Step 3: Create the execution role

The agent runtime needs an IAM role that grants it permission to invoke the model, write logs, call the gateway, and access memory. AgentCore assumes this role using a service principal with conditions that prevent confused deputy attacks:

```
const runtimeRole = new iam.Role(this, 'AgentRuntimeRole', {
  assumedBy: new iam.ServicePrincipal(
    'bedrock-agentcore.amazonaws.com'
  ).withConditions({
    StringEquals: {
      'aws:SourceAccount': this.account,
    },
    ArnLike: {
      'aws:SourceArn':
        `arn:aws:bedrock-agentcore:${this.region}:`
        + `${this.account}:runtime/*`,
```

```
    },
  }),
});

runtimeRole.addToPolicy(new iam.PolicyStatement({
  actions: [
    'bedrock:InvokeModel',
    'bedrock:InvokeModelWithResponseStream',
    'cloudwatch:PutMetricData',
    'logs:CreateLogGroup',
    'logs:CreateLogStream',
    'logs:PutLogEvents',
  ],
  resources: ['*'],
}));

tools.toolFunction.grantInvoke(runtimeRole);
dataBucket.grantReadWrite(runtimeRole);
```

The conditions on the trust policy limit which AgentCore resources can assume this role. Without them, any AgentCore runtime in the account could assume the role. The `aws:SourceArn` condition restricts assumption to runtimes in this account and region.

10.4 Step 4: Provision AgentCore Memory

Memory is provisioned as an independent resource before the runtime. Both the Strands and LangGraph implementations receive the memory ID through an environment variable and bind it at startup; they do not create memory resources themselves.

```
import { Memory, MemoryStrategy } from
  '@aws-cdk/aws-bedrock-agentcore-alpha';

const memory = new Memory(this, 'AgentCoreMemory', {
  memoryName: `agent_memory_${props.stage.name}`,
  description: 'Long-term memory for user preferences.',
  expirationDuration: Duration.days(30),
  memoryStrategies: [
    MemoryStrategy.usingUserPreference({
      name: 'user_preferences',
      description: 'Long-term preferences across sessions.',
      namespaces: ['/actors/{actorId}/preferences'],
    }),
  ],
});

memory.grantWrite(runtimeRole);
memory.grantRead(runtimeRole);
```

The memory strategy declares that this resource stores user preferences keyed by actor identifier. AgentCore Memory handles the vector index and retrieval. The agent code uses the memory ID to construct the checkpointer, the store, or the session manager depending on the framework, and the namespace `/actors/{actorId}/preferences` is where both the Strands session manager and the LangGraph pre-model hook write and search.

10.5 Step 5: Create the AgentCore Gateway

The gateway is the MCP endpoint that the agent uses at runtime. It advertises tool schemas, enforces IAM authentication on every tool invocation, and routes calls to the Lambda backend.

```
import {
  Gateway,
  GatewayAuthorizer,
  GatewayProtocol,
  MCPProtocolVersion,
  McpGatewaySearchType,
} from '@aws-cdk/aws-bedrock-agentcore-alpha';

const agentCoreGateway = new Gateway(scope,
  'AgentGateway', {
  gatewayName: `agent-gateway-${stageName}`,
  description: 'MCP gateway for agent tools.',
  authorizerConfiguration: GatewayAuthorizer.usingAwsIam(),
  protocolConfiguration: GatewayProtocol.mcp({
    supportedVersions: [
      MCPProtocolVersion.MCP_2025_06_18,
      MCPProtocolVersion.MCP_2025_03_26,
    ],
    searchType: McpGatewaySearchType.SEMANTIC,
    instructions:
      'Use these tools to retrieve data, trigger processing '
      + 'jobs, inspect results, and record governed decisions.',
  }),
});

agentCoreGateway.addLambdaTarget(
  'AgentToolsTarget', {
  gatewayTargetName: 'agent-tools',
  description: 'Domain tools via Lambda.',
  lambdaFunction: toolFunction,
  toolSchema: ToolSchema.fromInline(AGENT_TOOL_SCHEMA),
});

agentCoreGateway.grantInvoke(runtimeRole);
```

The gateway is not the Lambda function. The Lambda function is a target: it receives invocations from the gateway when a tool is called. The gateway owns the MCP protocol layer, handles tool discovery, and enforces SigV4 authentication. This means the agent code calls the gateway endpoint, not the Lambda directly. The gateway decouples the agent from the underlying implementation so a tool target can be a Lambda, a Fargate service, or any HTTP endpoint without changing the agent code.

The `grantInvoke(runtimeRole)` call allows the runtime container to call the gateway. Without it, the signed requests from the agent would fail with an authorisation error.

The runtime role also needs a specific gateway invocation permission:

```
runtimeRole.addToPolicy(new iam.PolicyStatement({
  actions: ['bedrock-agentcore:InvokeGateway'],
  resources: [
    agentCoreGateway.agentCoreGatewayArn,
```

```
    `${agentCoreGateway.agentCoreGatewayArn}`
    + `/gateway-endpoint/*`,
  ],
}));
```

10.6 Step 6: Package and deploy the Runtime

With the role, memory, and gateway in place, the runtime can be declared. The CDK `Runtime` construct packages the `agent/` directory as a Docker image and registers it with AgentCore:

```
import {
  AgentRuntimeArtifact,
  Runtime,
  RuntimeAuthorizerConfiguration,
  RuntimeNetworkConfiguration,
  ProtocolType,
} from '@aws-cdk/aws-bedrock-agentcore-alpha';
import { ecrAssets } from 'aws-cdk-lib';

const runtime = new Runtime(this, 'AgentRuntime', {
  runtimeName: props.stage.runtimeName,
  executionRole: runtimeRole,
  agentRuntimeArtifact: AgentRuntimeArtifact.fromAsset(
    path.join(__dirname, '..', 'agent'), {
      platform: ecrAssets.Platform.LINUX_ARM64,
    }
  ),
  networkConfiguration:
    RuntimeNetworkConfiguration.usingPublicNetwork(),
  protocolConfiguration: ProtocolType.HTTP,
  authorizerConfiguration:
    RuntimeAuthorizerConfiguration.usingIAM(),
  environmentVariables: {
    STAGE: props.stage.name,
    AWS_REGION: this.region,
    MODEL_ID: DEFAULT_MODEL_ID,
    GATEWAY_URL: agentCoreGateway.agentCoreGatewayUrl,
    GATEWAY_REGION: this.region,
    MEMORY_ID: memory.memoryId,
    MEMORY_REGION: this.region,
  },
});
```

Several details are worth examining.

`AgentRuntimeArtifact.fromAsset` builds a Docker image from the `agent/` directory and pushes it to ECR. The agent directory must contain a `Dockerfile` that installs dependencies and sets the container entry point to the agent application module. CDK handles the build and push during `cdk deploy`.

`environmentVariables` is the bridge between infrastructure and code. The agent reads the gateway URL, memory ID, and model identifier from environment variables. This keeps the same container image portable across stages: a personal stage, a gamma stage, and a production stage all use the same image but receive different environment variables at runtime.

`RuntimeAuthorizerConfiguration.usingIAM()` means that every caller of the runtime must present a valid SigV4 signature. Callers need the `bedrock-agentcore:InvokeAgentRuntime` permission for the runtime ARN.

10.7 Step 7: Add the proxy Lambda for browser clients

Browser applications cannot sign requests with SigV4 directly. The backend solves this with a runtime proxy Lambda that sits between the API Gateway and the AgentCore runtime:

```
const runtimeProxyFn = new lambda.Function(this,
  'RuntimeProxyFunction', {
  runtime: lambda.Runtime.PYTHON_3_11,
  handler: 'index.handler',
  code: lambda.Code.fromAsset(
    path.join(__dirname, '..', 'lambda', 'runtime-proxy')
  ),
  timeout: Duration.seconds(30),
  memorySize: 512,
  environment: {
    ALLOWED_RUNTIME_PATTERN: `^${props.stage.runtimeName}`,
    CORS_ALLOWED_ORIGINS: '*',
  },
});

runtimeProxyFn.addToRolePolicy(new iam.PolicyStatement({
  actions: ['bedrock-agentcore:InvokeAgentRuntime'],
  resources: [
    runtime.agentRuntimeArn,
    `${runtime.agentRuntimeArn}/runtime-endpoint/*`,
  ],
}));

const invokeResource = api.root
  .addResource('runtime')
  .addResource('invoke');

invokeResource.addMethod('POST',
  new apigateway.LambdaIntegration(runtimeProxyFn), {
    authorizationType: apigateway.AuthorizationType.IAM,
  }
);
```

The proxy Lambda accepts a request from the IAM-protected API Gateway endpoint, validates the runtime name against the `ALLOWED_RUNTIME_PATTERN` environment variable, constructs the `runtimeSessionId` from the request, calls `bedrock-agentcore:InvokeAgentRuntime`, parses the streaming response, and returns a JSON envelope to the caller.

The `ALLOWED_RUNTIME_PATTERN` check prevents callers from invoking arbitrary runtimes by constructing a different runtime name. The API Gateway endpoint is protected by IAM, so the browser application authenticates with IAM credentials (typically via Amazon Cognito) and the proxy does the SigV4 signing on its behalf.

This proxy pattern is not convenience wrapping. It is a policy enforcement point: it controls which runtimes can be invoked, how session identifiers are constructed, and how streaming output

is reduced to a response that the browser can parse.

10.8 Step 8: Deploy and verify

The stack is deployed with the CDK CLI:

```
# Install CDK dependencies
npm install

# Deploy to a personal stage
npm run deploy:personal

# Or use CDK directly
npx cdk deploy AgentBackendStack-personal
```

After deployment, CDK outputs the runtime ARN, the API URL, the gateway URL, and the memory ID. These values are the concrete links between the CDK infrastructure and the agent's environment variables.

A minimal smoke test verifies the runtime is alive without invoking any backend tools or the model:

```
import boto3, json, uuid

def invoke_agent(
    runtime_arn: str,
    region: str,
    prompt: str,
) -> str:
    client = boto3.client(
        "bedrock-agentcore", region_name=region
    )
    response = client.invoke_agent_runtime(
        payload=json.dumps(
            {"prompt": prompt}
        ).encode("utf-8"),
        runtimeSessionId=f"testuser___{uuid.uuid4()}",
        agentRuntimeArn=runtime_arn,
    )
    body = response.get("response") or response.get("body")
    return extract_response_text(iter_objects(body))

# Send a health prompt and verify the container is alive
text = invoke_agent(runtime_arn, region, prompt="health")
print(text) # expected: {"status": "ok", ...}
```

The health prompt takes the fast path in both agent implementations: it skips the model, the gateway tools, and the memory layer, and returns a static status object. This makes it safe to call frequently as a liveness check without incurring model costs. For a fuller integration test, send a real task prompt and use Langfuse to capture the trace. The evaluation chapter treats that trace as the foundation of diagnostic scoring.

10.9 What the runtime owns versus what it delegates

The deployment creates a clean separation of ownership across services.

Service	Responsibility
AgentCore Runtime	Packages and runs the agent container; manages invocation lifecycle and session routing
AgentCore Gateway	Exposes backend tools via MCP; enforces SigV4 authentication on every tool call; handles protocol versioning
AgentCore Memory	Stores conversational state, checkpoints, and long-term preferences; provides semantic retrieval
Lambda (tools)	Implements all domain operations; owns business logic and state persistence
DynamoDB	Persists typed records with a configurable TTL
EventBridge	Triggers scheduled or event-driven processing pipelines
API Gateway	Provides browser-facing ingress with IAM authentication and CORS
Lambda (proxy)	Enforces runtime name policy; translates between API Gateway and AgentCore invocation protocol
CloudWatch	Stores structured logs from all Lambda functions; enables query-based diagnostics
Secrets Manager	Stores runtime secrets so they are not baked into the container image

The model is surrounded by conventional cloud services. It does not touch infrastructure directly. When the agent calls a gateway tool, IAM governs the call, the gateway validates the schema, the Lambda runs the domain logic, and DynamoDB stores the result. The agent sees a tool response.

10.10 Strands and LangGraph in the same runtime

Both frameworks deploy to AgentCore using the same CDK stack. The only structural difference is which Python module in the `agent/` directory is set as the entry point in the Dockerfile. The CDK runtime artifact, the IAM role, the gateway, and the memory resource are identical in both cases.

This is the key property of the deployment pattern: it is framework-neutral. The runtime does not care whether the agent is a Strands `Agent` object or a LangGraph `create_react_agent` graph. It cares that the agent exposes an HTTP server on port 8080, accepts a JSON payload, yields events in the expected format, and responds to health checks.

The same memory ID works with the Strands `AgentCoreMemorySessionManager` and with the LangGraph `AgentCoreMemorySaver` and `AgentCoreMemoryStore`. The same gateway URL works with the Strands `GatewayToolPool` and with the LangGraph `MultiServerMCPClient`. The framework choice is an application-layer decision; the deployment layer does not need to change when that decision changes.

10.11 Summary

Deploying an agent to AgentCore follows a fixed order: tool layer first, then IAM role, then memory, then gateway, then runtime, then proxy. Each resource depends on resources created before it. The CDK stack encodes this dependency graph as TypeScript and `cdk deploy` resolves it into a set of CloudFormation change sets.

The agent code stays portable throughout. It reads its gateway URL, memory ID, and model identifier from environment variables. The same code runs locally against a test gateway, on a personal stage, and in production. The deployment layer supplies environment variables and handles authentication; the agent code handles reasoning. That separation is what allows the same assistant to run as a Strands agent or a LangGraph graph without modifying the infrastructure at all.

Chapter 11

Evaluation

Evaluating an agentic system requires choosing, at the outset, what the evaluation is intended to measure. The simplest approach is outcome-based evaluation: given a task, did the agent produce the correct final answer? Its limitation is that a correct answer carries no information about the path taken to reach it. An agent may succeed on a given instance through a sequence of steps that is inefficient, brittle, or unlikely to generalise.

Trajectory-based evaluation addresses this by treating the sequence of decisions as the object of study, not merely its outcome. Rather than scoring the final answer alone, the evaluator inspects the full sequence of tool calls, retrieval steps, and intermediate reasoning: whether the right tools were selected, whether they were invoked with valid arguments, and whether each step was necessary given the state of the task at that point. This approach is more demanding to instrument and score, but it produces a more diagnostic picture of system behaviour, one that can distinguish a robust agent from one that happens to succeed on the test distribution.

11.1 Traces

The practical foundation of both evaluation modes is the trace: a structured, replayable record of a single agent execution. A trace captures the user input, every model call, every tool invocation and its result, any retrieved context, the final answer, per-step latency, and accumulated cost. Without traces, evaluation is limited to comparing inputs and outputs; with them, it becomes possible to isolate the specific decision that caused a failure.

Traces must reflect the hierarchical structure of agent execution. When a parent agent delegates to a sub-agent, that nested execution should appear as a child span within the parent trace. When a tool call fails, the error should be attached to the corresponding observation rather than surfaced only at the top level. This structure is what makes a trace analytically useful rather than merely a log.

11.2 Scores

Evaluation results are represented as scores attached to traces or to individual spans within them. Scores can be produced by automated checks, by human annotators, or by LLM-based judges, and they can target different levels of the execution. At the outcome level, the relevant dimensions include whether the agent satisfied the user's objective, whether factual claims are correct, and whether assertions are grounded in retrieved evidence or tool outputs. At the trajectory level, relevant dimensions include whether the agent selected appropriate tools, whether arguments were

valid and complete, and whether tool results were correctly incorporated into subsequent reasoning. Orthogonal to both are efficiency scores, measuring whether the task was completed within acceptable cost and latency budgets, and safety scores, which assess whether the agent respected applicable permissions and constraints.

11.3 Datasets and Experiments

Langfuse supports input/expected-output datasets as the baseline format for offline evaluation. As the system is deployed and production traces accumulate, the dataset can be versioned and progressively enriched: traces flagged during online audit, where the agent failed or behaved unexpectedly, become new labelled examples in the next dataset version. This mechanism ensures that the evaluation suite tracks the actual failure modes encountered in production, and that fixes introduced in a new agent version are verified against the exact conditions that exposed the original defect.

11.4 Stratified Sampling with Unbalanced Weights

Production traffic is rarely uniform. In a typical deployment, the majority of conversations receive no explicit feedback, a small fraction receive a positive signal such as a thumbs-up, and a smaller fraction still receive a negative signal. Annotating a simple random sample of this traffic would leave the most informative cases, especially those where the agent visibly failed, severely underrepresented.

Stratified sampling addresses this by partitioning traffic into buckets and allocating annotation capacity to each bucket independently. In the feedback example, the natural partition is:

Bucket	**Meaning**
$h = \uparrow$	Thumbs up
$h = \downarrow$	Thumbs down
$h = \emptyset$	No feedback

Let W_h denote the true production share of bucket h, and let n_h denote the number of examples annotated within that bucket. For each bucket, the observed defect rate is:

$$\hat{p}_h = \frac{\text{number of defective examples in bucket } h}{n_h}$$

The overall production defect rate is then recovered by re-weighting each bucket estimate by its production share:

$$\hat{p}_{\text{overall}} = \sum_h W_h \hat{p}_h$$

This is the stratified sampling correction: because annotation overrepresents certain buckets, each bucket-level estimate must be scaled back to its true weight before aggregation. The approximate variance of this estimator is:

$$\text{Var}(\hat{p}_{\text{overall}}) = \sum_h W_h^2 \, \frac{\hat{p}_h(1 - \hat{p}_h)}{n_h}$$

and the resulting 95% confidence interval is:

$$\hat{p}_{\text{overall}} \pm 1.96 \sqrt{\sum_h W_h^2 \frac{\hat{p}_h(1-\hat{p}_h)}{n_h}}$$

A practical consequence of this formula is that uncertainty in the overall estimate is dominated by the largest production bucket. If the no-feedback population represents 98.5% of traffic, then $W_\emptyset^2$ is an order of magnitude larger than the weights of the other buckets, and the no-feedback variance term will dominate the confidence interval regardless of how many thumbs-down examples are annotated. Annotating high-signal buckets such as thumbs-down is valuable for diagnosing failure modes, but it does not substitute for adequate coverage of the no-feedback population when the goal is to estimate overall production performance reliably.

Numerical example

Suppose production traffic is distributed as follows:

Bucket	**Prod. share W_h**	**Annotated n_h**	**Defect rate $\hat{p}_h$**
Thumbs up	1.0%	200	2%
Thumbs down	0.5%	500	65%
No feedback	98.5%	2 000	8%

The weighted defect estimate is:

$$\hat{p}_{\text{overall}} = 0.010 \times 0.02 + 0.005 \times 0.65 + 0.985 \times 0.08 = 0.0823$$

The standard error is:

$$SE = \sqrt{0.010^2 \cdot \frac{0.02 \times 0.98}{200} + 0.005^2 \cdot \frac{0.65 \times 0.35}{500} + 0.985^2 \cdot \frac{0.08 \times 0.92}{2000}} \approx 0.00598$$

The 95% confidence interval is therefore [7.05%, 9.40%]. The no-feedback term contributes virtually all of the variance, illustrating that increasing the thumbs-down annotation budget would narrow the interval only marginally.

11.5 LLM-as-a-Judge

Some evaluation criteria resist rule-based scoring: helpfulness, clarity, groundedness, and policy compliance each require a degree of interpretive judgment. LLM-based judges can operationalise these criteria at scale, but they introduce systematic biases that must be accounted for, including verbosity bias, position bias, and a tendency toward self-preference when the judge and the agent under evaluation share the same base model. Mitigation requires explicit, criterion-anchored rubrics; eliciting a chain of evidence before requesting a numerical score; enforcing structured output; running at low temperature; and calibrating judge scores against human annotations on a held-out subset.

LLM judges should not displace deterministic checks where deterministic checks are applicable. Conformance to a JSON schema should be verified by a schema validator. Functional correctness of generated code should be verified by executing tests. The role of an LLM judge is to handle the residual criteria that deterministic methods cannot reach.

11.6 Failure Taxonomy

The diagnostic value of traces is substantially higher when failures are categorised consistently. A working taxonomy for agentic systems covers three failure modes: incorrect tool usage, which encompasses both wrong tool selection and malformed input parameters; incorrect output, which covers both hallucination and wrong format; and guardrails violation. Labelling failures according to such a taxonomy transforms a collection of individual errors into a distribution of failure modes, which in turn makes it possible to prioritise improvements and measure whether interventions are effective.

11.7 Offline and Online Evaluation

Offline evaluation runs a fixed dataset before deployment and functions as a gate: a promotion is blocked if quality falls below a defined threshold. Online evaluation samples live production traffic after deployment and functions as a monitor: it detects distribution shift, novel user behaviour, tool failures, and model drift that fixed datasets do not anticipate. The two modes are complementary and a mature system requires both. Offline evaluation gives deployment confidence; online evaluation gives operational visibility. Human annotation periodically calibrates the automated scoring pipeline, and production trace mining continuously supplies new candidate items for the offline dataset, closing the loop between the two modes.

11.8 Summary

Agent evaluation is not a single procedure but a system of interlocking practices: choosing between outcome and trajectory evaluation, instrumenting traces, defining scored dimensions, maintaining a versioned dataset, deploying LLM judges with appropriate mitigations, and operating both offline and online pipelines. The connecting principle is that every production failure should become a labelled example, every labelled example should become an evaluation, and every evaluation should become a deployment gate.

Conclusion

The four-layer architecture this book describes: intelligence, action, memory, knowledge, and security, represents the foundation of agentic systems.

The field is moving fast. Open-weight models are closing the gap with closed frontier models on the dimensions that matter most for agentic tasks, enabling a democratisation of the technology that will broaden access to production-grade agent capabilities across organisations and research communities.

The harder problem is reliability and governance: building systems that act autonomously on behalf of people and organisations with the same engineering rigour applied to traditional machine learning software. Evaluation, observability, and principled deployment pipelines are the infrastructure that makes that possible. They are also the least glamorous part of the work, and the part most often deferred.

The reader who has followed this book to its end has the foundation to build agents that hold up not only in proof of concept but in deployment.